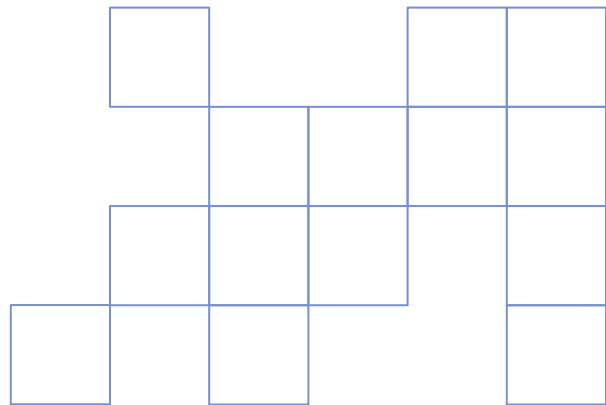


Dorm Companion **“Dormio”**



Juggling Student Life



Organizing Schedules

Conflicts with school schedule

Evolving Schedules



Managing Finances

Managing expenses

Managing Bills



Meal Planning

Long term planning

Balancing nutrition



Dorm Maintenance

Complication in scheduling requests

No instant updates



Chore Splitting

Manage between all residents

Track progress of others

Our Solution



Features Overview

- Managing Schedule by adding events with visual conflict warning
- Bill splitting amongst dorm residents
- Maintenance Request directly connected with dorm maintenance team
- Sharable chore list editable amongst dorm residents
- Trackable Expenses and budget list with visualization

Key Deliverables

1.



Functional MVP containing all of the apps core features

- e. g. Meal planning, Finances, Chore listing

2.



Project Documentation

- e.g. User guides and user workflow overviews
- Installation and setup instructions

3.



App Testing Results

- E.g. list of features that are tested with Status (pass rate)
- Description of test cases used

4.



Database

- Updates are held locally when database is down
- Handling of user errors when interacting with database features



Project Vision – Dormio

Purpose

Dormio is a dorm companion app created by Tekwave to simplify student life by **centralizing** key day-to-day tasks (budgeting, meal planning, and maintenance management).

Vision Statement

To reduce stress and improve organization for students living in residence while supporting staff in managing dorm operations **efficiently**.



Project Vision Cont'd

Key Features

- Automated budget management
- Meal planning with dorm-wide coordination
- Maintenance request tracking and updates
- Customizable and flexible design that fits any dorm system

Stakeholders

- Primary: Students living in residence
- Secondary: Residence administrators and staff

Current Focus (MVP)

- Core features only: budgeting, meal planning, maintenance
- Out of scope:
 - 3rd-party software integrations
 - In-app messaging or chat features

Project Plan - Scope



- **In Scope:**
 - Customizable Meal Manager
 - Budget Management System (Incl. Bill Splitting)
 - Maintenance Requests System (Student & Staff)
 - All Required Documentation (Plan, Charter, etc.)
- **Out of Scope (Risk Management):**
 - Third-Party Software Integration
 - Group Communication Features

3. Scope

Define the sum total of all of its products and their requirements or features.

In Scope	Out of Scope
User and Staff Documentation - Provision of user guides and technical documentation that clearly illustrate application flow, features, and system architecture	Live Payment Processing - Dormio will not process, integrate with, or initiate any online transactions.
User Management - Implementation of secure user accounts and profiles for both student and residence staff	Handling of Sensitive Information - Dormio will not store sensitive data such as financial credentials.
Customizable Meal Manager - functionality allowing students to select, save, and modify a personalized, meal plan based on preferences.	Migration of Data - there will be no automated tools to migrate existing student information into Dormio
Budget Management System - A tool for personal finance tracking, including manual transaction entry and bill splitting functionality among assigned roommates.	Third-Party Software Integration - Dormio will not integrate third-party software.
Maintenance Requests System - Allows students to submit and track maintenance requests, and enables Residence staff to manage and update request status.	Group Communication Features - features such as in-app group chat or direct messaging between residents are out of scope



Project Plan - Assumptions

We Assume the Following Will Hold True:

- **Reliable Internet Access** for all student users.
- **Dorm Administration will adopt** the new Maintenance System.
- **Security measures are sufficient** to protect sensitive student data.
- **Project scope will remain fixed** after today's approval.
- **Team members will maintain committed hours** (Resource Availability).

1. **Consistent User Connectivity:** We assume student residents will have reliable and consistent internet access within the dormitories, as the app is designed to function as a centralized, real-time hub.
2. **Administrative System Adoption:** We assume the Dorm Administration team will fully adopt and integrate the new Maintenance Request System, and provide any necessary access permissions for the system to interface with residence workflows.
3. **Data Security Sufficiency:** We assume the security measures implemented are fully sufficient and compliant to adequately protect all sensitive student information, particularly financial and personal data.
4. **Positive User Engagement:** We assume the target student user group will be willing to consistently and frequently use the new app for managing daily tasks like finances and meal planning.
5. **Technical Stack Stability:** We assume the chosen technology stack will remain accessible (free or within budget) and stable throughout the development and testing lifecycle.
6. **Unchanged Project Scope:** We assume the finalized set of features and requirements will remain fixed after the initial approval, preventing any scope creep that would jeopardize the project deadline.
7. **Resource Availability:** We assume all core team members will maintain the agreed-upon availability and commitment needed to complete their assigned tasks without major unforeseen personal or academic conflicts.



Project Plan - Dependencies

We Need These Prerequisites Complete:

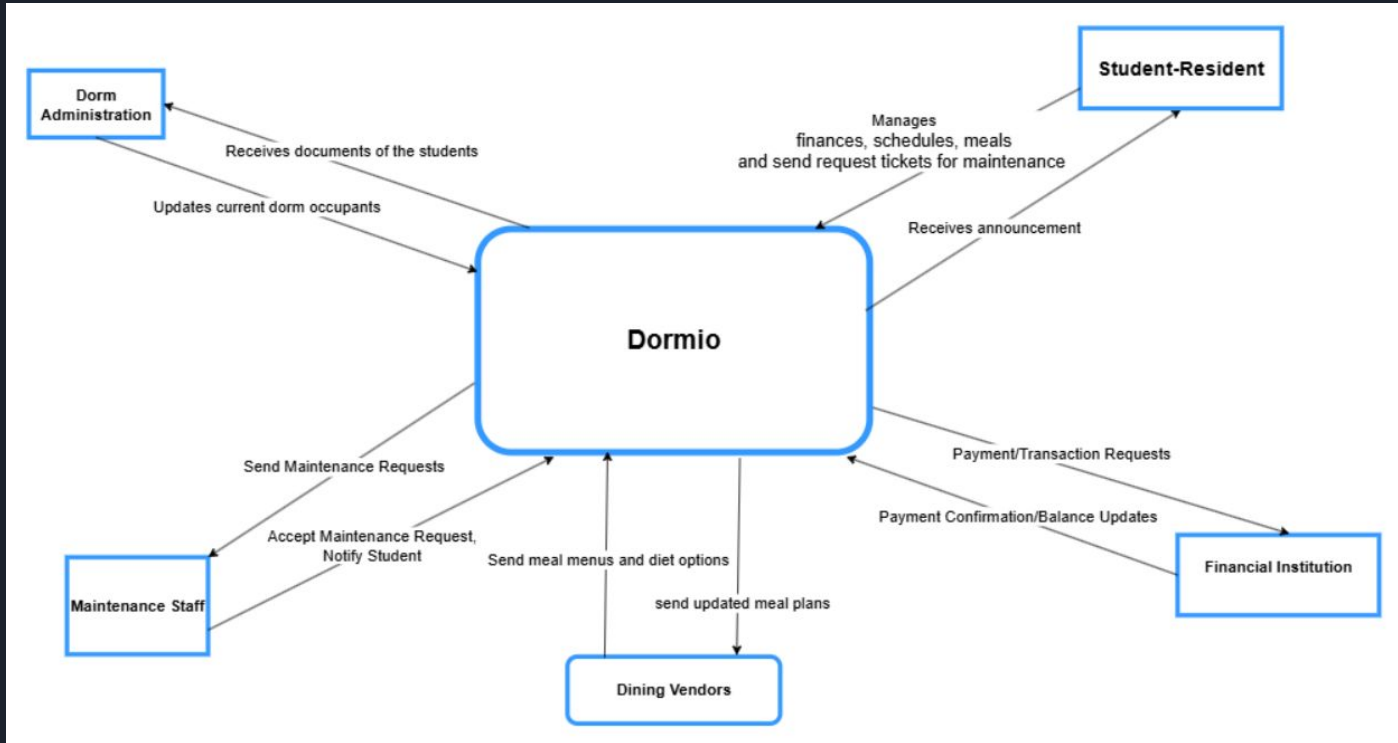
- **Database connected and seeded** with initial data.
- **VCS (GitHub) System Setup** for team collaboration.
- **Database Schemas.**
- **Internal Data Structure** finalized (local vs. database storage).
- **Final sign-off on Wireframes**
- **Final Approval** from Residence Staff on maintenance workflow.

6. Dependencies

The following are the internal and external dependencies that will have to be acknowledged and addressed;

1. **Database Connection & Seeding:** Our software is dependent on the population of the database. All feature functionality and testing are blocked until the database is successfully connected and populated with initial test data.
2. **Version Control System (VCS) Setup:** Our team will push to the shared Git repository so that each member is working on the latest version of the software. All collaborative coding efforts are dependent on this system being properly set up.
3. **Finalized Design Documents:** Successful implementation of features is dependent on the final sign-off of the Wireframes for all features and the Designing database schemas and relationships.
4. **Team Member Output:** Implementation of complex features is dependent on the successful completion of foundational work by other team members
5. **Administrative Access Credentials:** Final testing and deployment of the Maintenance Request System requires cooperation and the provision of necessary access credentials from the Residence Staff.
6. **Required External Approvals:** The implementation of features impacting staff (like the Maintenance Request System) is dependent on final approval and sign-off from the Residence Staff/Management on the proposed workflow and data collection.
7. **Resource Allocation:** The project timeline is dependent on all team members maintaining their commitment to the hours allocated in the RAM and Gantt chart.

Business Context Diagram





Requirements Scope Statements

HLR#	Description	Priority (H, M, L)
HLR01	Students must be able to manage their finances using a budget management tool.	H
HLR01	The dorm administrator must be able to receive and track student maintenance request	H
HLR01	Students must be able to submit and track maintenance requests by completing a ticket form with a description of the issue.	H
HLR04	The students must be able to customize their meal plan based on their daily schedule.	M
HLR05	The student must be allowed to view any charges associated with the usage of the dorm facilities	M
HLR06	The application must allow the student to split their bill when managing	L



Risk Management

- Team member leaves the group or misses deadline
 - Redistribution of work
 - Escalate the problem
- Lack of technical knowledge
 - Assign tasks by strengths or provide help
- Paid third party services (e.g. database)
 - Use of local storage
- Loss of source code and files
 - Create backups by Github or external hard drive
- Integration issues
 - Weekly builds

- Features not working to another member
 - Standardized dependencies
- Unclear feature description
 - Making sure that each meeting the members are present
 - Following the gantt chart
- Scope of the project might be greater than planned
 - Simplify the feature
 - Create a buffer by 5 days before deadline
 - Update the plan

Task Listing



The documentation and the designing phase

- Task A to E

Reference	Tasks	Duration	Dependency
A	Decide on tech stack	4 h	
B	Decide what data to store	5 h	A
C	Design of User roles	8 h	B
D	Creation of wireframe	3 d	C
E	Database design and relationships	2 d	D



Task listing



Start of basic implementations

- Tasks F to J

Reference	Tasks	Duration	Dependency
F	Connection to database	1 d	E
G	Github Set-up (app & web app)	4 h	
H	CRUD implementation	5 d	F
I	UI design (admin)	6 d	H
J	UI design Mobile	4 d	H

Task listing



Features implementation

- K to N



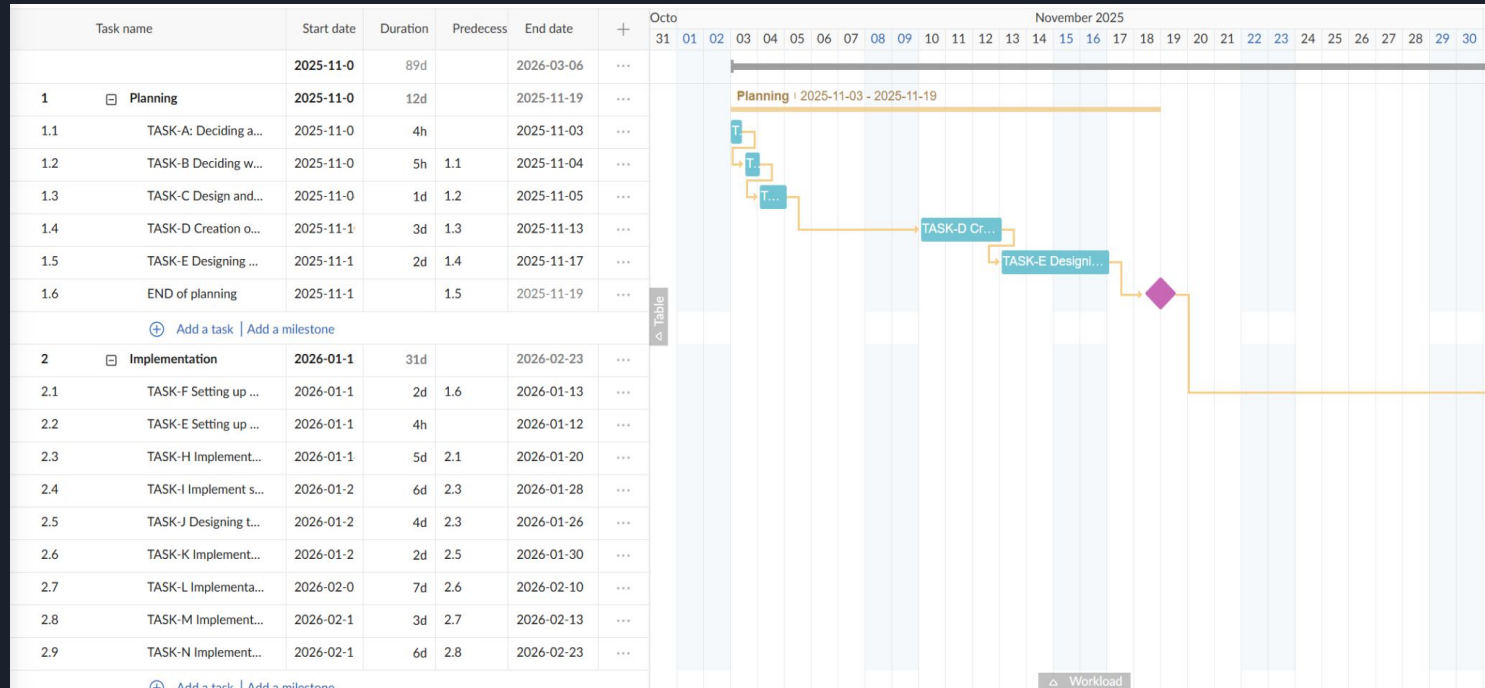
Debug and testing

- O to P

K	Add budget data charts	1 d	J
L	Features implement with logic restriction	7 d	K
M	Chore manager & CSV export	3 d	L
N	REST API integration to mobile app	6 d	J - M

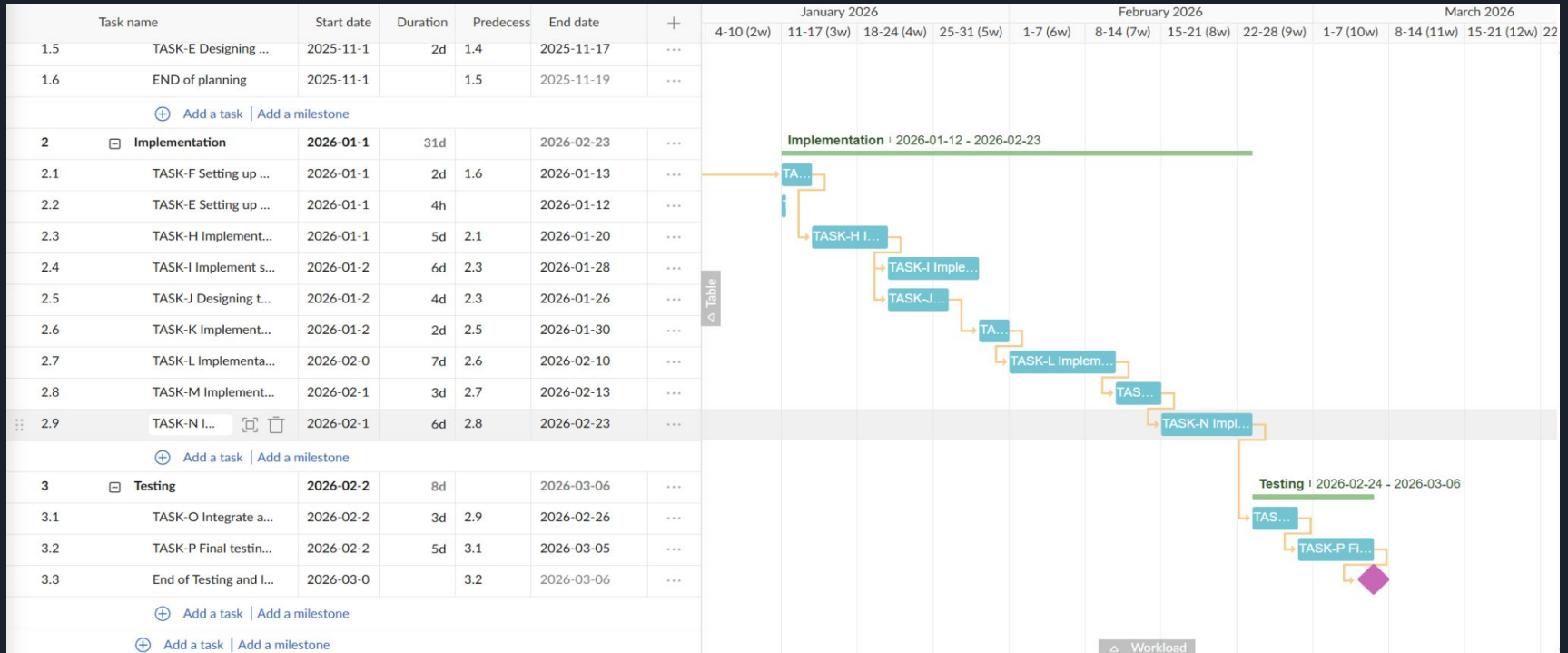
O	Integrate and test all features to dashboard	3 d	N
P	Final testing and Debugging	5 d	P

Gantt Chart for Sprint 3 to 4



Link: <https://app.ganttpro.com/#/project/1761265068674/gantt>

Gantt Chart sprint 5 to 8



Milestones

Major Activity or Milestone	Estimated Milestone Target date	Owner/Reviewer Team Members
Deciding on tech stack finalized	November 3, 2025	Bryan Paul Yabut, Patrick Sherwin Millares
Data storage structure planned (local vs database)	November 4, 2025	Ibrahim Abdillahi, Bryan Paul Yabut
User roles and permissions completed	November 5, 2025	Patrick Solamillo, Ibrahim Abdillahi
Wireframes created for Admin site and Mobile app	November 13, 2025	Yousuf Salihi, Patrick Sherwin Millares
Database schema and relationships finalized	November 17, 2025	Ibrahim Abdillahi



Milestones

Major Activity or Milestone	Estimated Milestone Target date	Owner/Reviewer Team Members
Database connection setup and seeded	January 13, 2026	Patrick Sherwin Millares
GitHub repository setup and project initialized	January 12, 2025	Bryan Paul Yabut
CRUD operations implemented	January 20, 2026	Team
Admin site UI simplified and designed	January 28, 2026	Patrick Sherwin Millares, Yousuf Salihi
Mobile app UI completed	January 26, 2026	Yousuf Salihi

Milestones

Major Activity or Milestone	Estimated Milestone Target date	Owner/Reviewer Team Members
Budget Management data visualization added	January 30, 2026	Yousuf Salihi
Scheduling, meal planning, and maintenance request features implemented	February 10, 2026	Patrick Sherwin Millares, Ibrahim Abdillahi
Chore Manager and CSV export completed	February 13, 2026	Bryan Paul Yabut, Patrick Solamillo
REST API integrated with mobile app	February 23, 2026	Yousuf Salihi, Patrick Sherwin Millares
Unified dashboard integration	February 26, 2026	Team
Final testing and debugging	February 05, 2026	Team
Final presentation and demo	March 06, 2026	Team

Responsibility Assignment Matrix

Project Team Responsibilities

Project Name:	Dormio				
Project Manager:	Laily Ajellu				
Task	Bryan Paul Yabut	Ibrahim Abdillahi	Patrick Sherwin Millares	Patrick Solamillo	Yousuf Salihi
Deciding on appropriate tech stack	P		S		
Deciding what data needs to be stored locally and on the database	S	P			
Designing and creating user roles & permissions		P		S	
Creating wireframes for Admin site & Mobile app			S		P
Designing database schema & relationships		P	S		
Setting up database connection & seeding data		S	P		
Setting up GitHub repositories & initializing project	P			S	
Implementing CRUD operations for all features	S	S	P	S	
Implementing simplified Admin UI			P		S
Designing Mobile App UI			S		P
Implementing Budget Management visualization					P
Implementing scheduling, meal planning & maintenance features		S	P		
Implementing Chore Manager & CSV export	P			S	
REST API integration with Mobile app			P		S
Integrate & test all features into a unified dashboard	S	S	P	S	S
Final testing & debugging	S	S	S	S	P
P = Primary (Responsible) S = Secondary (Support)					

Thank You For Listening

